

TP BAC sujet n°18

CHAPITRE

Dictionnaire

QUESTION 1:

CONSIGNES

- Écrire une fonction `temperature_moyenne(zone, donnees)` qui :
- Prend en paramètres une chaîne `zone` (nom d'un archipel) et un tableau `donnees` de dictionnaires représentant les relevés
 - Renvoie la température moyenne (float) pour cette zone
 - Renvoie `None` si la zone n'a aucun relevé

FONCTION PYTHON

```
def temperature_moyenne(zone, donnees):  
    somme=0  
    nb_temp=0  
    for i in donnees:  
        if i['zone']==zone:  
            somme+=i['temperature']  
            nb_temp+=1  
    if nb_temp==0:  
        res=None  
    else:  
        res=somme/nb_temp  
    return res
```

Création d'une fonction température moyenne avec en paramètre le nom d'un archipel et le dictionnaire des données

- Initialisation d'une variable `somme` à 0 qui va contenir la somme des températures
- Initialisation d'une variable `nb_temp` à 0 qui va contenir le nombre de températures
- Boucle `for` pour parcourir tout les éléments du dictionnaire
- Vérifie que l'élément correspond à celui recherché
- Si l'élément correspond on ajoute la température à `somme`
- Si l'élément correspond on ajoute 1 à `nb_temp` (ce qui correspond à une température en plus dans `somme`)
- Si le nombre de température est vide :
- On donne `None` comme valeur à `res`
- Sinon:
- On donne la somme des températures divisé par le nombre de températures à `res`
- On renvoie `res`

EXPLICATION

- La fonction `temperature_moyenne` calcule la température moyenne pour une zone donnée à partir d'une liste de relevés.
- Elle commence par initialiser une somme des températures et un compteur à zéro, puis parcourt l'ensemble des données.
- Pour chaque relevé, elle vérifie si la zone correspond à celle recherchée ; si c'est le cas, elle ajoute la température à la somme et incrémente le compteur.
- Une fois tous les relevés parcourus, elle vérifie si aucun élément ne correspond à la zone : dans ce cas, elle renvoie `None`.
- Sinon, elle calcule et renvoie la moyenne en divisant la somme des températures par le nombre de relevés correspondants.

QUESTION 2:

CONSIGNES

- Écrire une fonction `detecter_anomalies(zone, seuil, donnees)` qui :
- Prend en paramètres une zone, un seuil (float), et la liste `donnees`
 - Calcule la température moyenne de la zone
 - Renvoie la liste des dates où la température s'écarte de plus de seuil degrés (en valeur absolue) de cette moyenne
 - Renvoie une liste vide si la zone n'existe pas

FONCTION PYTHON

```
def detecter_anomalies(zone, seuil, donnees):  
    temp_moy=temperature_moyenne(zone, donnees)  
    anomalies=[]  
    if temp_moy==None:  
        return []  
    else:  
        for i in donnees:  
            if i['zone']==zone and abs(i['temperature']-temp_moy)>seuil:  
                anomalies.append(i['date'])  
        return anomalies
```

Création d'une fonction `detecter_anomalies` avec en paramètre une zone, un seuil (floattant) et le dictionnaire des données

- On initialise `temp_moy` à la température moyenne grâce à la fonction précédente
- On crée une liste vide pour stocker les anomalies
- Si la moyenne des température n'existe pas: (cas de base)
- On renvoie un tableau vide
- Sinon:
- On parcourt chaque élément du dictionnaire
- Si l'élément appartient à la zone et si l'écart à la moyenne dépasse le seuil:
- Une fois la boucle finie, on renvoie la liste des anomalies

EXPLICATION

- La fonction `detecter_anomalies` sert à identifier les dates où la température d'une zone s'écarte significativement de sa moyenne.
- Elle commence par calculer la température moyenne de la zone, puis initialise une liste vide pour stocker les anomalies.
- Si la zone n'existe pas dans les données, la fonction renvoie immédiatement une liste vide.
- Sinon, elle parcourt tous les relevés et sélectionne ceux qui appartiennent à la zone étudiée et dont la température diffère de la moyenne de plus que le seuil fixé.
- Pour chacun de ces relevés, elle ajoute la date correspondante à la liste des anomalies. Enfin, elle renvoie cette liste.

QUESTION 3:

CONSIGNES

- _____ Compléter les trois fonctions de test, situées à la fin du fichier, pour la fonction `evolution_par_decennie` en utilisant le jeu de données fourni. _____
- _____ 1. `test_zone_inexistante()` : Tester une zone qui n'existe pas _____
- _____ 2. `test_une_seule_decennie()` : Tester une zone avec données sur une seule décennie _____
- _____ 3. `test_plusieurs_decennies()` : Tester une zone avec données sur plusieurs décennies _____
- _____ Vos tests doivent permettre d'identifier le bug présent dans le code. _____

FONCTION PYTHON

```
def test_zone_inexistante():
    resultat=evolution_par_decennie('Zone_inexistante', donnees_test)
    assert resultat=={}
```

- _____ Fonction `test_zone_inexistante` sans paramètres _____
- _____ On donne comme valeur à resultat l'appel de la fonction `evolution_par_decennie` avec une zone qui n'existe pas _____
- _____ On vérifie que le résultat est un dictionnaire vide _____

```
def test_une_seule_decennie():
    resultat=evolution_par_decennie('Marquises', donnees_test)
    assert len(resultat)==1
    assert 2020 in resultat
    assert resultat[2020]==26.0
```

- _____ Fonction `test_une_seule_decennie` sans paramètres _____
- _____ On donne comme valeur à resultat l'appel de la fonction `evolution par decennie` avec la zone 'Marquises' _____
- _____ On vérifie que le résultat contient qu'une seule décennie _____
- _____ On vérifie que la décennie 2020 est présente dans resultat _____
- _____ On vérifie que la température moyenne de la décennie 2020 est bien de 26.0 _____

```
def test_plusieurs_decennies():
    resultat=evolution_par_decennie('Societe', donnees_test)
    assert len(resultat)>=2
    assert resultat[2010]==27.0
    assert resultat[2020]==28.5
```

- _____ Fonction `test_plusieurs_decennies` sans paramètres _____
- _____ On donne comme valeur à resultat l'appel de la fonction `evolution_par_decennie` avec la zone 'Societe' _____
- _____ On vérifie que le résultat contient plusieurs decennies _____
- _____ On vérifie que la température moyenne de la décennie 2010 est bien de 27.0 _____
- _____ On vérifie que la température moyenne de la décennie 2020 est bien de 28.5 _____

QUESTION 4:

CONSIGNES

Après avoir identifié le bug grâce à vos tests, corriger la fonction `evolution_par_decennie`.

FONCTION PYTHON

```
1 def evolution_par_decennie(zone, donnees):
2     releves_zone = [r for r in donnees if r['zone'] == zone]
3     if not releves_zone:
4         return {}
5     temperatures_par_decennie = {}
6     for releve in releves_zone:
7         annee = int(releve['date'].split('-')[0])
8         decennie = (annee // 10)*10
9         if decennie not in temperatures_par_decennie:
10             temperatures_par_decennie[decennie] = []
11             temperatures_par_decennie[decennie].append(releve['temperature'])
12     moyennes = {}
13     for decennie, temperatures in temperatures_par_decennie.items():
14         moyennes[decennie] = round(sum(temperatures) / len(temperatures), 2)
15     return moyennes
```

Correction ligne 8

Multipliation de decennie par 10

Decennie sans correction → 201

Decennie avec correction → 2010